SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1004	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	**Total Marks:**	25
Student Name:	Pradhap J	**Reg. No.:**	1924260246
Date:	10-10-2026	**Duration:**	60 minutes

Code of Conduct

I Pradhap J / 1924260246 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

Annexure A – Case Study Analysis

Problem Statement 38: Digital Research Project Management Platform

1. Understanding the Problem Statement

Problem in my own words. Academic research involves researchers, supervisors, milestones, literature, datasets, experiments, meetings and publications. These are normally handled in separate tools such as email, chat apps, spreadsheets and shared drives. As a result, progress is hard to see, deadlines slip, files get duplicated or lost, and meeting decisions are not followed up. A single platform is needed to manage a research project from proposal to publication.

Objectives of the proposed system

- Create and manage research projects with objectives, scope and team members.
- Define milestones and assign tasks with deadlines and statuses.
- Track progress through dashboards and timelines, with alerts for delays.
- Store and version documents, literature references and datasets in one repository.
- Record supervisor meetings: minutes, decisions and action items.
- Track publication activities (draft, submitted, under review, accepted).
- Provide role-based access and generate periodic progress reports.

Scope

In scope: a web-based system for project, milestone, task, document, meeting and publication management, with dashboards, notifications, role-based access and reports. *Out of scope:* running experiments, statistical or data-analysis tools, and journal submission portals (only the submission status is tracked).

2. Key Stakeholders and Challenges

Stakeholder	Roles and responsibilities	Key challenges
Researcher / Student	Creates and updates tasks, uploads documents, logs progress, attends meetings	Missed deadlines, scattered files, unclear priorities
Supervisor / Guide	Sets milestones, reviews work, gives feedback, approves submissions	No real-time visibility of progress; feedback is lost in email
Co-researchers	Collaborate on tasks, datasets and documents	Version confusion, unclear ownership of work
Research Coordinator / Department	Monitors all projects, reviews reports	Manual and inconsistent reporting
System Administrator	Manages users, roles, backups and security	Access control and protection of unpublished research
Publication bodies (indirect)	Review and accept or reject papers	Status must be tracked across several venues

Major challenges: poor coordination between students and supervisors, lost or duplicate documents, forgotten meeting decisions, delays discovered too late, and no centralized documentation.

3. Analysis of the Current Approach

Existing process. Communication runs through email and chat apps, tasks and deadlines are tracked in spreadsheets or notebooks, files sit in shared drives or personal devices, and meeting notes are kept on paper or in chat messages. Reports are prepared by hand.

Strengths	Limitations
Free and familiar to everyone Flexible, so no training is needed Quick to start for small projects	No unified view of project progress Version control problems and duplicate files No automatic delay alerts Meeting action items are not tracked Weak access control and no audit trail Manual, time-consuming report preparation

Gaps to be addressed: no centralized repository; no link between milestones and tasks; no automated notifications; no publication tracking; no analytics or dashboards; and no secure, role-based collaboration.

4. Proposed Solution Using Software Engineering Principles

Solution design. A role-based, three-tier web application: a React front end, a REST backend (Node.js or Spring Boot) with JWT authentication, a relational database (MySQL/PostgreSQL) for structured data, and cloud file storage for documents and datasets. Email and in-app notifications are handled by a scheduler service.

Major functional modules

Module	Description
User and Role Management	Registration, login, roles (Student, Supervisor, Admin)
Project Management	Objectives, scope, team members, project status
Milestone and Task Management	Create milestones, assign tasks, deadlines, status updates
Document and Dataset Repository	Upload, versioning, literature references, search
Meeting Management	Schedule meetings, minutes, decisions, action items
Publication Tracker	Draft, submitted, under review, accepted or rejected
Dashboard and Progress Tracking	Timeline, percentage completion, delay indicators
Notification Module	Deadline and meeting reminders by email and in-app
Reports Module	Periodic progress reports exported as PDF

Functional requirements

- Secure registration, login and role assignment.
- Create projects, milestones and tasks, and assign them to members.
- Upload, version and download documents and datasets.
- Log meetings with minutes and action items; track action completion.
- Track publication status through its stages.
- Send automatic reminders for upcoming and overdue items.
- Display progress dashboards and generate reports.

Non-functional requirements

- **Performance:** pages load within 3 seconds under normal load.
- **Availability:** at least 99% uptime.
- **Security:** encrypted storage and transport, role-based access.
- **Usability:** simple, responsive interface for desktop and mobile.
- **Scalability:** supports hundreds of projects and concurrent users.

Software engineering principles applied

Principle	How it is incorporated
Modularity	Independent modules with defined REST interfaces; each can be built and tested separately
Scalability	Layered architecture, stateless APIs, database indexing and cloud deployment that can scale out
Maintainability	Clean code standards, documentation, Git version control, automated unit tests
Reliability	Input validation, error handling, regular backups and recovery procedures
Usability	Role-specific dashboards, consistent navigation, clear status colors and alerts
Security	Password hashing, JWT authentication, role-based access control, audit logs, HTTPS

5. Justification of the Chosen Methodology: Agile (Scrum)

Selected framework: Agile using Scrum, with two-week sprints, a product backlog, sprint reviews and retrospectives.

Why it is suitable

- Requirements will change as supervisors and students give feedback, which Scrum handles through backlog re-prioritization.
- The modules are largely independent, so they can be delivered incrementally: Sprint 1 login and projects; Sprint 2 milestones and tasks; Sprint 3 documents and meetings; Sprint 4 publications, dashboard and reports.
- A working product is available after the first sprints, so real users can review it early.
- Waterfall is too rigid for evolving requirements, and Spiral adds risk-analysis overhead that is heavier than this project needs.

Support for development and maintenance

- Sprint reviews and retrospectives catch problems early and improve the process.
- A prioritized backlog keeps the team on the most valuable features.
- Continuous testing and integration keep quality high.
- After release, fixes and enhancements fit naturally into new sprints without restructuring the project.

6. Expected Outcomes and Limitations

Expected benefits

- A single source of truth for each research project.
- Stronger supervisor-student coordination and faster feedback.
- Early detection of delays through alerts and dashboards.
- Centralized, versioned documentation and meeting records.
- Faster, automated progress reporting.

How the solution addresses the challenges

Challenge	Solution
Scattered tools and files	One central repository with versioning
Missed deadlines	Milestone-linked tasks with automated alerts
Forgotten meeting decisions	Recorded minutes and trackable action items
Poor progress visibility	Real-time dashboards and timelines
Manual reporting	Auto-generated periodic reports

Risks and limitations

- Users may resist moving away from familiar tools; training and a simple interface are needed.
- Requires internet access and depends on users updating information regularly.
- Unpublished research data demands strong security and privacy controls.
- Scope creep during development must be controlled through the backlog.

Future enhancements

- AI-based delay prediction and workload suggestions.
- Plagiarism checking and citation tools.
- Integration with Google Scholar, Zotero and GitHub.
- A mobile app and video-meeting integration.